

Algorithm Trade Equation Sheet

1. Adjusting SMA Value

In real-time applications or simulations, it is often necessary to update a Simple Moving Average (SMA) based on a hypothetical or new price without re-summing the entire historical window. This is computationally efficient and useful for "what-if" analysis.

Variable Definitions

Let:

- $n \in \mathbb{N}$ = the window length or time period for the Simple Moving Average.
- c_i = the historical closing price at index i , where c_1 represents the most recent closed price.
- P_{adj} = the hypothetical or new adjusted price replacing c_1 .
- SMA_n = the current Simple Moving Average for period n .
- $_{adj}SMA_n$ = the new adjusted Simple Moving Average incorporating P_{adj} .

Derivation

Given the current Simple Moving Average for a period n :

$$SMA_n = \frac{1}{n} \sum_{i=1}^n c_i$$

If we wish to replace the most recent closing price c_1 with an adjusted price P_{adj} to find the adjusted average $_{adj}SMA_n$, we define:

$$_{adj}SMA_n = \frac{(\sum_{i=1}^n c_i) - c_1 + P_{adj}}{n}$$

Substituting $\sum_{i=1}^n c_i = n \cdot SMA_n$:

$$_{adj}SMA_n = \frac{n \cdot SMA_n - c_1 + P_{adj}}{n}$$

Dividing through by n , we arrive at the adjustment formula:

$$_{adj}SMA_n = SMA_n + \frac{P_{adj} - c_1}{n}$$

Interpretation

This formula demonstrates that the impact of a price change on the total average is proportional to the magnitude of the change ($P_{adj} - c_1$) divided by the window length n .

- As n increases, the average becomes more "weighted" by history and less sensitive to individual price adjustments.

- This calculation is vital for determining how a current "live" price affects the moving average before the candle actually closes.

2. Calculating SMA Crossover Price

Variable Definitions

Let:

- $n, m \in \mathbb{N}$ where $n < m$ (short-term and long-term periods).
- x = the target price required for a crossover.
- c_i = the historical closing prices, where c_1 is the most recent closed price.

Derivation of the Crossover Price

A crossover occurs at the moment the short-term average equals the long-term average. To find the price x that causes this in the next period, we set the two averages equal:

$$\frac{(\sum_{i=2}^n c_i) + x}{n} = \frac{(\sum_{i=2}^m c_i) + x}{m}$$

To isolate x , we multiply both sides by the product of the denominators ($n \cdot m$):

$$m \left(\sum_{i=2}^n c_i + x \right) = n \left(\sum_{i=2}^m c_i + x \right)$$

Expanding the terms:

$$m \sum_{i=2}^n c_i + mx = n \sum_{i=2}^m c_i + nx$$

Grouping the terms containing x on the left-hand side and the constant sums on the right-hand side:

$$mx - nx = n \sum_{i=2}^m c_i - m \sum_{i=2}^n c_i$$

Factoring out x :

$$x(m - n) = n \sum_{i=2}^m c_i - m \sum_{i=2}^n c_i$$

The final solution for the limit cross price x is:

$$x = \frac{n \sum_{i=2}^m c_i - m \sum_{i=2}^n c_i}{m - n}$$

Calculating Price Difference (d) using Current SMA

Instead of solving for the absolute price x , we can solve for the difference d relative to the current closing price c_1 , such that $x = c_1 + d$.

Starting from the equality of the two averages:

$$\frac{(\sum_{i=2}^n c_i) + (c_1 + d)}{n} = \frac{(\sum_{i=2}^m c_i) + (c_1 + d)}{m}$$

We can merge c_1 back into the summations to represent the **current** price sums (including the most recent close):

$$\frac{(\sum_{i=1}^n c_i) + d}{n} = \frac{(\sum_{i=1}^m c_i) + d}{m}$$

Recognizing that $\sum_{i=1}^k c_i = k \cdot \text{SMA}_k$, we substitute the current SMA values:

$$\frac{n \cdot \text{SMA}_n + d}{n} = \frac{m \cdot \text{SMA}_m + d}{m}$$

Multiplying both sides by nm to clear the denominators:

$$m(n \cdot \text{SMA}_n + d) = n(m \cdot \text{SMA}_m + d)$$

Expanding the terms:

$$mn \cdot \text{SMA}_n + md = nm \cdot \text{SMA}_m + nd$$

Grouping the terms with d on the left-hand side:

$$md - nd = nm \cdot \text{SMA}_m - mn \cdot \text{SMA}_n$$

Factoring out d and the constant nm :

$$d(m - n) = nm(\text{SMA}_m - \text{SMA}_n)$$

Solving for d :

$$d = \frac{nm(\text{SMA}_m - \text{SMA}_n)}{m - n}$$

Consequently, the limit cross price is:

$$x = c_1 + \frac{nm(\text{SMA}_m - \text{SMA}_n)}{m - n}$$

Conclusion and Simulation Logic

The derivation of d provides a mathematically precise "limit cross price" (x) that represents the exact threshold for a trend shift. In a backtesting or live simulation environment, this allows for a much more granular estimation of execution prices compared to standard "signal-on-close" methods.

Instead of assuming a trade occurs at the next period's Open or Close, the simulation can determine if the crossover price x was actually reachable within the current candle's price action.

Intra-Candle Execution Estimation

To achieve a higher-fidelity simulation, the following threshold-based logic is applied to determine the effective **cross_price** based on the calculated limit cross price x :

- **Long Position Side (Bullish Cross):**

If the cross is triggered by price moving upward, we check if x was reachable within the candle's range:

$$\text{cross_price} = \begin{cases} x & \text{if } x \geq \text{Low} \\ \text{Open} & \text{if } x < \text{Low (Gap/Immediate)} \end{cases}$$

- **Short Position Side (Bearish Cross):**

Conversely, if the cross is triggered by price moving downward:

$$\text{cross_price} = \begin{cases} x & \text{if } x \leq \text{High} \\ \text{Open} & \text{if } x > \text{High (Gap/Immediate)} \end{cases}$$

By utilizing this logic, simulations avoid the "lag" associated with signal-on-close methods, resulting in performance metrics that more closely reflect real-time market dynamics.

3. SMA Trend (Consecutive Runs) Maintenance

A "Simple Trend" is defined by the slope of the SMA. To maintain a trend for n periods, the price must not cross a specific threshold that would cause the SMA to reverse direction.

Trend Maintenance Price

To keep the current SMA ($extSMA_n$) greater than or equal to the previous value ($preextSMA_n$), we solve for the required closing price P_{maint} . Using the adjustment formula:

$$SMA_n = preSMA_n + \frac{c_1 - c_{1+n}}{n}$$

To maintain the trend (where $\Delta SMA = 0$), the maintenance price is derived as:

$$P_{maint} = c_1 + n(preSMA_n - SMA_n)$$

Trend Logic and Execution

The trend is considered "confirmed" if the simple trend direction has been consistent for a window of L periods (where $L = \text{TREND_LEN}$).

Intra-Candle Trend Execution

For simulation accuracy, if a trend is active ($res = 1$ for Long or -1 for Short), the execution price is determined by whether the market "gapped" beyond the maintenance price:

- **Long Trend ($res = 1$):**

$$\text{Price} = \begin{cases} \text{Open} & \text{if Low} > P_{maint} \\ P_{maint} & \text{otherwise} \end{cases}$$

- **Short Trend ($res = -1$):**

$$\text{Price} = \begin{cases} \text{Open} & \text{if High} < P_{\text{maint}} \\ P_{\text{maint}} & \text{otherwise} \end{cases}$$

4. Dynamic Retracement (Trailing Stop) Update

A trailing stop-loss based on a retracement percentage typically requires tracking the entry price and the peak price reached during the trade. This section derives an optimized recursive formula for updating the exit price.

Variable Definitions

Let:

- P_{entry} = Initial entry price.
- $M_{\text{old}}, M_{\text{new}}$ = The previous and current maximum prices reached since entry.
- R = Retracement percentage (expressed as a decimal, e.g., 0.10 for 10%).
- $E_{\text{old}}, E_{\text{new}}$ = The previous and current calculated exit prices.

Derivation of the Recursive Update

The standard formula for a retracement exit price is:

$$E = P_{\text{entry}} + (M - P_{\text{entry}})(1 - R)$$

To find the relationship between the old exit and the new exit when the maximum price updates from M_{old} to M_{new} , we look at the difference:

$$E_{\text{new}} - E_{\text{old}} = [P_{\text{entry}} + (M_{\text{new}} - P_{\text{entry}})(1 - R)] - [P_{\text{entry}} + (M_{\text{old}} - P_{\text{entry}})(1 - R)]$$

The P_{entry} terms cancel out:

$$E_{\text{new}} - E_{\text{old}} = (1 - R)(M_{\text{new}} - P_{\text{entry}} - M_{\text{old}} + P_{\text{entry}})$$

$$E_{\text{new}} - E_{\text{old}} = (1 - R)(M_{\text{new}} - M_{\text{old}})$$

Solving for E_{new} , we arrive at the recursive adjustment formula:

$$E_{\text{new}} = E_{\text{old}} + (1 - R)(M_{\text{new}} - M_{\text{old}})$$

Interpretation and Efficiency

This formula demonstrates that the exit price moves upward by a fraction $(1 - R)$ of the growth in the maximum price.

- **Computational Efficiency:** The bot no longer needs to store or reference the P_{entry} variable to update the stop; it only needs the most recent exit price and the change in the high.
- **Sensitivity:** If R is 0.20 (a 20% retracement), the exit price trails the new high at 80% of the expansion rate.

Execution Logic

In a live environment, the update only triggers if a new high is established ($M_{\text{new}} > M_{\text{old}}$):

1. **Check for New High:** If $\text{High}_{\text{current}} > M_{\text{old}}$, then:

$$\Delta M = \text{High}_{\text{current}} - M_{\text{old}}$$

$$E_{\text{new}} = E_{\text{old}} + \Delta M(1 - R)$$

2. **Check for Exit Trigger:** If $\text{Price}_{\text{current}} \leq E_{\text{new}}$, execute exit.

5. Exponential Moving Average (EMA) Rounding Errors and Convergence Window

Calculating an Exponential Moving Average (EMA) truncated to n historical periods introduces a truncation/rounding error due to ignoring prior history beyond period n . This section derives the exact error bound and isolates the minimum required lookback window n needed to achieve precision up to k decimal places.

Variable Definitions

Let:

- $S \in \mathbb{R}^+ =$ the smoothing factor (typically $S = 2$).
- $N \in \mathbb{N} =$ the time period/length of the EMA.
- $\alpha =$ the smoothing multiplier, defined as $\alpha = \frac{S}{1+S}$.
- $c_i =$ the historical closing price at index i , where c_0 is the current price.
- $\text{EMA}_i =$ the true Exponential Moving Average at period i .
- $\text{SEED} =$ the initial seed value used to approximate historical values prior to index n .
- $E =$ the seed deviation error, defined as $E = \text{EMA}_{n+1} - \text{SEED}$ (approximated as $\text{SEED} \cdot 0.1$).
- $k \in \mathbb{N} =$ the desired decimal precision threshold (i.e., the first insignificant decimal place 10^{-k}).
- $n \in \mathbb{N} =$ the number of lookback terms required to meet the precision threshold.

Derivation

1. Recursive Expansion

The standard recursive definition for the current EMA (EMA_0) is given by:

$$\text{EMA}_0 = \alpha \cdot c_0 + (1 - \alpha) \cdot \text{EMA}_1$$

Expanding this relationship recursively for historical prices $c_0, c_1, c_2, \dots$ reveals the infinite series representation:

$$\text{EMA}_0 = \alpha \sum_{i=0}^{\infty} (1 - \alpha)^i c_i$$

2. Truncation and Seed Error

When truncating the calculation to n historical terms, the infinite tail is replaced by a seed value SEED at period $n + 1$:

$$\text{EMA}_0 = \alpha \sum_{i=0}^n (1 - \alpha)^i c_i + (1 - \alpha)^{n+1} \cdot \text{EMA}_{n+1}$$

Because EMA_{n+1} is unknown in practice, using SEED introduces a total error Err :

$$\text{Err} = (1 - \alpha)^{n+1} \cdot (\text{EMA}_{n+1} - \text{SEED})$$

Substituting the seed error term $E = \text{EMA}_{n+1} - \text{SEED}$:

$$\text{Err} = (1 - \alpha)^{n+1} \cdot E$$

3. Isolating Minimum Lookback Window (n)

To ensure the error is lower than our precision threshold 10^{-k} , we set:

$$10^{-k} = (1 - \alpha)^{n+1} \cdot E$$

Dividing both sides by E :

$$\frac{10^{-k}}{E} = (1 - \alpha)^{n+1}$$

Taking the natural logarithm ($\ln$) of both sides:

$$\ln \left(\frac{10^{-k}}{E} \right) = \ln ((1 - \alpha)^{n+1})$$

Using logarithmic properties:

$$-k \ln(10) - \ln(E) = (n + 1) \ln(1 - \alpha)$$

Solving for n :

$$n + 1 = \frac{-k \ln(10) - \ln(E)}{\ln(1 - \alpha)}$$

$$n = \frac{-k \ln(10) - \ln(E)}{\ln(1 - \alpha)} - 1$$

Practical Application

To guarantee accuracy up to k decimal places in live trading systems:

1. Estimate $E \approx \text{SEED} \cdot 0.1$.
2. Compute n using the formula above and round up to the nearest integer ($\lceil n \rceil$).
3. This guarantees that truncating the historical price array at n periods will not impact trade signal execution beyond 10^{-k} .